

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: INTERNET PROGRAMMING**  
**COURSE CODE: CSA4305**

### COURSE OUTCOME COVERED

**CO4:** Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

### QUESTIONS:

**Total Marks: 25**

#### 1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

#### 2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

#### 3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

#### 4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

## 5: PHP Validation

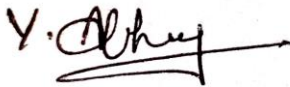
Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

### **Code of Conduct:**

*I Y. Abhay- 192411219 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

### **Signature of the student:**



### **RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4 – Exemplary                                                   | Level 3 – Proficient                      | Level 2 – Developing         | Level 1 – Beginning             |
|---------------------------|-----------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20%       | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30%       | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20%       | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10%       | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20%       | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |

# 1. XML Parsing Algorithm

## Pseudocode: Parse XML Document

**Input:** XML file containing user/product details

**Output:** Extracted XML data displayed in readable format

### Psuedocode:

ALGORITHM XML\_PARSING

START

1. Specify the path of the XML document.
2. Load the XML document into memory.
3. Check whether the XML document is loaded successfully.

IF loading fails THEN  
    Display "Error loading XML file"  
    STOP  
END IF

4. Access the root element of the XML document.
5. Identify all required child elements under the root.
6. Traverse each child node one by one.
7. FOR each node in the XML document DO

    Identify the type of node.

    IF the node contains child elements THEN  
        Traverse its child nodes recursively.  
    END IF

    IF the node contains required data THEN  
        Extract the element name.  
        Extract the element value.  
    END IF

    Store or display the extracted data.

END FOR

8. Format the extracted information in a readable form.

9. Display the required user/product details.

10. Release the XML document resources.

END

### **Example Output**

Product Details

-----

ID : 101

Name : Laptop

Price : 55000

Stock : 20

### **Key Point**

The algorithm uses **hierarchical traversal** so that parent, child, and nested elements are handled correctly.

## 2. MVC Workflow

### Pseudocode: MVC Request Processing

**Input:** User request

**Output:** Response displayed to the user

#### Pseudocode:

ALGORITHM MVC\_WORKFLOW

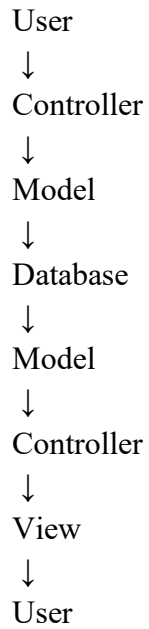
START

1. User sends a request through the web browser.
2. Web application receives the request.
3. Send the request to the Controller.
4. Controller identifies the requested operation.
5. Controller validates the request data.
6. Controller calls the appropriate Model method.
7. Model processes the business logic.
8. Model connects to the database if required.
9. Model executes the required database operation.
10. Database returns the requested data.
11. Model processes the database result.
12. Model sends the result back to the Controller.
13. Controller selects the appropriate View.
14. Controller passes the required data to the View.
15. View generates the response page.
16. Send the generated response to the user's browser.

17. Browser displays the result.

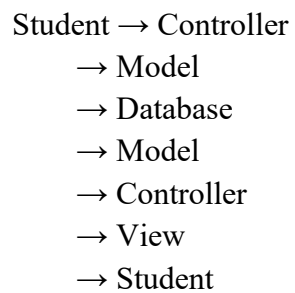
END

### **MVC Flow**



### **Example**

For a student searching for a book:



### 3. JSP Request Flow

#### **Pseudocode: JSP Request–Response Flow**

**Input:** Client request

**Output:** Dynamic JSP response

#### **Psuedocode:**

ALGORITHM JSP\_REQUEST\_FLOW

START

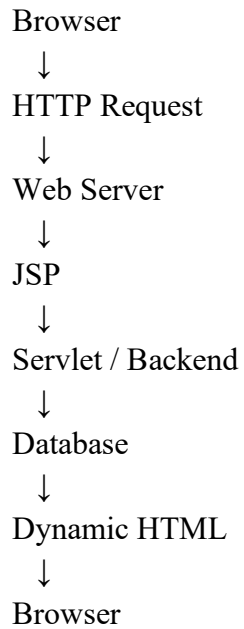
1. User enters a URL or submits a form.
2. Browser sends an HTTP request to the web server.
3. Web server receives the request.
4. Identify the requested JSP page.
5. Check whether the JSP page is already compiled.
6. IF JSP is not compiled THEN  
    Translate JSP into Servlet.  
    Compile the generated Servlet.  
END IF
7. Servlet Container executes the JSP-generated Servlet.
8. JSP/Servlet receives the request parameters.
9. Validate and process the input.
10. IF backend processing is required THEN  
    Call Servlet or business component.  
    Connect to database if required.  
    Execute required operation.  
    Retrieve the result.  
END IF
11. Generate dynamic HTML content using the retrieved data.
12. Send the generated HTML response to the web server.

13. Web server sends the response to the browser.

14. Browser displays the dynamic webpage.

END

### **JSP Flow**



### **Important Point**

A JSP is ultimately **translated into a Servlet and executed by the Servlet container.**



## 4. Database Retrieval

### Pseudocode: Retrieve Data from Database

**Input:** SQL SELECT query

**Output:** Retrieved records

ALGORITHM DATABASE\_RETRIEVAL

START

1. Import the required database connectivity components.

2. Specify:

    Database URL

    Username

    Password

3. Establish a connection with the database.

4. Check whether the connection is successful.

    IF connection fails THEN

        Display "Database connection failed"

        STOP

    END IF

5. Create a statement or prepared statement.

6. Write the required SELECT SQL query.

7. Execute the SQL query.

8. Store the returned records in a Result Set.

9. Check whether records are available.

10. WHILE records are available DO

    Read the current record.

    Extract the required fields.

    Display the values in readable format.

Move to the next record.

END WHILE

11. Close the Result Set.

12. Close the Statement.

13. Close the Database Connection.

14. Handle any database exceptions.

END

### **Database Flow**

Application



Database Connection



SQL Query



Database



Result Set



Process Records



Display Data

### **Example Output**

| ID  | Name     | Price |
|-----|----------|-------|
| 101 | Laptop   | 55000 |
| 102 | Keyboard | 1500  |
| 103 | Mouse    | 800   |

## 5. PHP Validation

### Pseudocode: PHP Form Validation

**Input:** User registration form

**Output:** Validated form or error messages

ALGORITHM PHP\_FORM\_VALIDATION

START

1. Receive the form data submitted by the user.

2. Initialize an empty error message list.

3. Read:

    Name

    Email

    Password

    Confirm Password

4. Check whether the Name field is empty.

    IF Name is empty THEN

        Add "Name is required" to error list.

    END IF

5. Check whether the Email field is empty.

    IF Email is empty THEN

        Add "Email is required" to error list.

    ELSE

        Check whether the email format is valid.

        IF email format is invalid THEN

            Add "Invalid email address" to error list.

        END IF

    END IF

6. Check whether the Password field is empty.

    IF Password is empty THEN

        Add "Password is required" to error list.

    ELSE

        Check password length.

```
    IF password does not satisfy requirements THEN
        Add "Invalid password" to error list.
    END IF
END IF
```

#### 7. Compare Password and Confirm Password.

```
    IF they are different THEN
        Add "Passwords do not match" to error list.
    END IF
```

#### 8. Check the error list.

```
    IF errors exist THEN
        Display all appropriate error messages.
        Do not submit the form.
    ELSE
        Accept the form.
        Process/store the validated data.
        Display "Registration Successful".
    END IF
```

END

### Validation Flow

